

Deep Reinforcement Learning for Long Term Profit Optimization in Thai Stock Markets Using Multi-Source Data

BY

Nathawat Youngyim
Kunach Kijsanayothin

SUBMITTED IN PARTIAL FULFILLMENT OF

THE REQUIREMENTS FOR GRADUATION IN 2026

KAMNOETVIDYA SCIENCE ACADEMY (KVIS)

Project Title	Deep Reinforcement Learning for Long Term Profit Optimization in Thai Stock Markets Using Multi-Source Data
By	Nathawat Youngyim
	Kunach Kijsanayothin
Field of Study	Computer Science
Project Advisor	Prompong Pakawanwong
Academic Year	2026

ABSTRACT

This study examines a completed deep reinforcement learning (DRL) pilot for Thai stock portfolio allocation and completed high-performance computing (HPC) experiments supporting reproducible execution. The observed dataset contains ADVANC, AOT, CPALL, KBANK, and PTT over 968 trading dates from 4 January 2021 to 30 December 2024. A chronological validation pilot compared proximal policy optimization (PPO) and advantage actor–critic (A2C) with equal-weight, mean-variance, and buy-and-hold baselines. Separate BistKA experiments measured vector-environment throughput, deterministic environment replay, and storage-safe execution of PPO, A2C, DDPG, TD3, and SAC.

PTT buy-and-hold achieved the highest reported annualized return-to-volatility ratio (0.405480); A2C and PPO produced negative cumulative returns and ratios of -0.434028 and -0.439532. One vector environment reached 353.0773 samples per second with 600.711 MiB maximum peak resident memory, whereas 16 environments reached 82.9607 samples per second and 8,737.996 MiB. Two independent subprocesses reproduced the same 549-step trajectory hash. The five-algorithm smoke completed in 33 minutes 20 seconds using two CPUs and produced five complete manifests, final models, and retention receipts with no checkpoint paths. Returned output fell from 275,209,516 to 137,777,145 bytes, a 49.94% reduction.

The historical pilot did not show DRL outperforming its strongest simple baseline. The HPC results demonstrate that measured resource calibration, deterministic environment checks, and storage-safe retention can strengthen reproducibility while reducing storage. Financial findings remain descriptive because they come from one validation split; smoke results establish executability, not algorithm quality.

Keywords: deep reinforcement learning, Thai stocks, portfolio allocation, high-performance computing, reproducibility, chronological validation

ACKNOWLEDGEMENTS

The authors thank their advisor, Prompong Pakawanwong, for guidance throughout the research process. They also acknowledge Kamnoetvidya Science Academy for providing the research environment and the BistKA Mini-HPC cluster used for computational calibration and controlled experiments.

The authors further thank the instructors and technical staff who supported cluster access, storage troubleshooting, and reproducible computational practice. Their assistance made it possible to distinguish measured scientific and engineering findings from unsupported performance claims.

Nathawat Youngyim
Kunach Kijsanayothin

TABLE OF CONTENTS

ABSTRACT i

ACKNOWLEDGEMENTS ii

LIST OF FIGURES v

LIST OF TABLES vi

1 INTRODUCTION 1

1.1 Background . . . 1
1.2 Research Problem . . . 2
1.3 Research Questions . . . 2
1.4 Scope and Contributions . . . 2

2 LITERATURE REVIEW 3

2.1 Portfolio Theory and Transparent Baselines . . . 3
2.2 Deep Reinforcement Learning Algorithms . . . 3
2.3 Temporal Evaluation and Backtest-Selection Risk . . . 4
2.4 Uncertainty and Multiple Comparisons . . . 4
2.5 Reproducible DRL and HPC Execution . . . 4
2.6 Literature Synthesis and Research Gap . . . 5

3 METHODOLOGY 6

3.1 Completed Experimental Design . . . 6
3.2 Observed-Market Data and Chronological Split . . . 7
3.3 Features and Leakage Controls . . . 8
3.4 Portfolio Environment, Algorithms, and Baselines . . . 8
3.5 Portfolio Metrics . . . 9
3.6 HPC Calibration and Deterministic Replay . . . 9
3.7 Storage-Safe Five-Algorithm Smoke . . . 10

4 DISCUSSION 11

4.1 Summary of Completed Observations 11
4.2 Historical Pilot Results 12
4.3 One Vector Environment Was Fastest and Most Memory-Efficient . . . 13
4.4 Five Algorithms Satisfied the Storage-Safe Artifact Contract 15
4.5 Storage-Safe Retention Halved Returned Output 15
4.6 Financial and Computational Interpretation 16
4.7 Limitations 17

5 CONCLUSION **18**
5.1 Summary of Findings 18
5.2 Scientific Conclusion 18

REFERENCES **20**

LIST OF FIGURES

3.1 Chronological split of the observed five-stock panel. Rows equal dates multiplied by five tickers. 8

4.1 Reported annualized return-to-volatility ratios in the completed historical validation pilot. The values come from one preserved validation table. 13

4.2 Observed throughput and sampled peak resident memory across vector-environment settings. Throughput error ranges show the minimum and maximum of three repeats. 14

4.3 Complete returned output inventories before and after storage-safe retention. The comparison measures artifact storage, not investment performance. 16

LIST OF TABLES

3.1 Completed experimental components included in this paper. 6
3.2 Observed-market cohort used by the completed experiments. 7
3.3 Descriptive portfolio metrics used in the completed pilot. 9

4.1 Summary of completed observations. 11
4.2 Real-OHLCV pilot results on one chronological validation split. Returns are decimals. 12
4.3 BistKA vector-environment calibration with three repeats per setting. . 14
4.4 Observed scheduler accounting and artifact validation for the five-algorithm smoke. 15

Chapter 1

INTRODUCTION

1.1 Background

Portfolio allocation requires a decision maker to balance return, risk, diversification, and trading costs over time. Classical portfolio theory formalizes the trade-off between expected return and variance [11], while risk-adjusted measures provide a way to compare strategies whose returns have different variability [12]. These ideas remain important even when the allocation rule is learned by a nonlinear model: a complex policy is useful only if it performs credibly against transparent alternatives.

Deep reinforcement learning treats portfolio management as sequential decision-making. At each trading date, an agent observes market and portfolio information, selects an allocation, and receives a reward that can incorporate return, turnover, and drawdown. Frameworks such as FinRL illustrate how market observations, trading constraints, portfolio actions, and backtesting can be integrated into this formulation [3]. DRL is attractive because it can represent nonlinear policies, but stochastic optimization and repeated model selection also create substantial risks of unstable or overfit results [1, 17].

High-performance computing is relevant because financial DRL experiments are computationally repetitive. Seeds, algorithms, and environment settings must be evaluated under consistent conditions, while intermediate outputs and software provenance must be retained. More hardware does not automatically improve a workload, however. Process overhead, memory duplication, storage pressure, and scheduling behavior can dominate when the environment is small. Consequently, HPC resource choices must be measured on the actual application rather than inferred from core count alone.

1.2 Research Problem

The central scientific problem is whether added model complexity produces useful portfolio behavior on observed Thai equity data. A convincing answer requires two forms of evidence. The first is financial evidence: learned policies must be compared with simple baselines on chronologically separated data. The second is computational evidence: the implementation must execute reproducibly, use resources efficiently, and preserve enough artifacts for independent inspection.

The selected five-stock cohort is intentionally narrow. It provides an observed-market pilot rather than a claim about the full Stock Exchange of Thailand. The completed performance experiment contains one historical validation split, so its values are descriptive rather than population-level estimates. The completed HPC experiments answer a different question: whether the software environment, resource configuration, and storage-retention mechanism behave as intended under measured conditions.

1.3 Research Questions

This paper addresses three questions using only completed experiments:

1. On the completed real-OHLCV validation split, how did PPO and A2C compare with equal-weight, mean-variance, and buy-and-hold baselines?
2. How did the number of parallel vector environments affect observed training throughput and process-tree memory on BistKA?
3. Could the five implemented DRL algorithms execute through a deterministic, storage-safe HPC path while preserving auditable final artifacts?

1.4 Scope and Contributions

The study covers five Thai equities from 4 January 2021 through 30 December 2024 and reports four completed contributions: a frozen validation comparison of PPO and A2C against transparent baselines; measured throughput and peak memory for 1, 2, 4, 8, and 16 vector environments; matching 549-step trajectories from two fixed-seed subprocesses; and a five-algorithm storage-safe smoke with a measured returned-output reduction. Financial and engineering findings remain separate: validation evaluates portfolio behavior, whereas calibration, replay, and smoke evaluate computational behavior.

Chapter 2

LITERATURE REVIEW

2.1 Portfolio Theory and Transparent Baselines

Markowitz introduced mean-variance portfolio selection as an explicit balance between expected return and portfolio variance [11]. The framework remains a natural classical comparator for an allocation model, but its estimated weights can be sensitive to sampling error. DeMiguel, Garlappi, and Uppal showed that evaluated mean-variance extensions did not consistently outperform the simple $1/N$ portfolio out of sample [10]. Equal weighting is therefore a demanding and interpretable benchmark rather than a trivial alternative.

Sharpe formalized a ratio of mean differential return to its standard deviation [12]. The completed implementation reports a related but not identical quantity: compounded annualized return divided by annualized volatility under a zero-risk-free-rate convention. This paper calls that quantity the *reported annualized return-to-volatility ratio* and defines it explicitly in Chapter 3.

2.2 Deep Reinforcement Learning Algorithms

FinRL frames automated trading as a sequential learning problem that combines market states, actions, transaction constraints, and backtesting [3]. The algorithms used in the completed experiments represent several actor–critic families. PPO constrains policy updates with a clipped surrogate objective [4]. A2C is a synchronous actor–critic implementation in the lineage of the asynchronous A3C architecture [5]. DDPG extends deterministic actor–critic learning to continuous action spaces [6]; TD3 reduces function-approximation and overestimation error through paired critics and delayed updates [7]; and SAC adds entropy regularization to off-policy learning [8].

Stable-Baselines3 supplies standardized implementations of these methods [9]. Standardized software reduces avoidable implementation differences, but it does not imply that any algorithm should dominate on Thai equities. The original algorithm papers mainly establish learning mechanisms on control or game benchmarks; their citation supports method provenance rather than financial suitability.

2.3 Temporal Evaluation and Backtest-Selection Risk

Financial observations are ordered and dependent. Randomly mixing future and past observations can leak information and produce an unrealistically favorable evaluation. Cerqueira, Torgo, and Mozetič found that out-of-sample estimation strategies are important for non-stationary time series and that repeated holdout methods can improve performance estimation [2]. The present study therefore preserves chronological order.

Chronological splitting alone does not eliminate selection bias. Bailey and colleagues demonstrate that repeated strategy search can overfit historical backtests and produce apparently strong strategies by chance [1]. This concern is particularly relevant to DRL, where training stochasticity and hyperparameter choice can generate many candidate policies. The completed pilot contains one validation split and is consequently interpreted as descriptive evidence rather than a definitive out-of-sample ranking.

2.4 Uncertainty and Multiple Comparisons

Bootstrap resampling provides a general approach to estimating uncertainty when an analytic sampling distribution is inconvenient [14]. Holm's sequentially rejective procedure controls family-wise error across multiple hypothesis tests [13]. These methods establish important statistical principles, but neither source alone validates independent resampling of serially dependent daily returns. Because the completed pilot contains one preserved validation table and no validated resampling analysis, this paper reports exact descriptive metrics without confidence intervals or adjusted significance claims.

2.5 Reproducible DRL and HPC Execution

Deep reinforcement learning results can vary with random seeds, hyperparameters, environment details, and reporting choices [17]. Reproducible computational research therefore requires precise records of data, configuration, software versions, random seeds, and the relationship between intermediate artifacts and reported results [16]. These prac-

tices are especially important on a shared cluster, where the scheduler, runtime modules, and filesystem form part of the experimental environment.

SLURM provides a general mechanism for resource allocation and job scheduling on Linux clusters [15]. In this study, BistKA scheduler records supply elapsed time, CPU allocation, memory requests, and exit status, while experiment manifests and checksums capture application-level provenance. Scheduler completion and scientific validity are treated as distinct conditions.

2.6 Literature Synthesis and Research Gap

The literature supplies portfolio baselines, continuous-control algorithms, temporal-evaluation principles, statistical tools, and reproducibility guidance. It does not determine whether PPO or A2C should outperform simple strategies on this fixed Thai-equity cohort. It also does not specify the most efficient vectorization level or storage-retention policy for this particular software and hardware combination. The paper addresses this narrow practical gap by pairing a completed observed-market pilot with measured HPC calibration, deterministic environment replay, and storage-safe execution evidence. It does not claim priority over all Thai-market DRL research.

Chapter 3

METHODOLOGY

3.1 Completed Experimental Design

The study consists of three completed experimental components, summarized in Table 3.1. The components share an observed five-stock data pipeline but answer different questions. The performance pilot evaluates portfolio metrics. The calibration and replay experiments evaluate resource behavior and environment repeatability. The storage-safe smoke evaluates whether all five algorithm paths complete while satisfying an artifact-retention contract.

Table 3.1: Completed experimental components included in this paper.

Component	Completed comparison	Scientific role
Real-OHLCV pilot	PPO and A2C versus four transparent baseline families on one chronological validation split	Descriptive portfolio-performance evidence
HPC calibration and replay	Five vector-environment settings, three repeats each; two independent fixed-seed trajectories	Resource selection and environment repeatability
Storage-safe smoke	One 2,048-timestep run for each of PPO, A2C, DDPG, TD3, and SAC	Executability, scheduler accounting, and artifact-retention evidence
Storage comparison	Complete returned inventories from checkpoint-writing and storage-safe smoke runs	Observed storage effect of the retention policy

3.2 Observed-Market Data and Chronological Split

The observed universe contains `ADVANC.BK`, `AOT.BK`, `CPALL.BK`, `KBANK.BK`, and `PTT.BK`. The panel contains 4,840 rows: 968 dates for each of five tickers from 4 January 2021 through 30 December 2024. The associated SET-index context contains 965 observed dates. Automated readiness checks recorded 15 passes, three disclosed warnings, and zero failures. The warnings consisted of ten deterministically normalized OHLC rows, three stock dates without a same-day SET-index observation, and constant scaffold fields that were not interpreted as economic signals.

Table 3.2: Observed-market cohort used by the completed experiments.

Item	Observed definition
Stock universe	ADVANC, AOT, CPALL, KBANK, and PTT Yahoo Finance symbols
Stock period	4 January 2021 through 30 December 2024
Panel size	4,840 rows; 968 dates per ticker; five tickers
Market context	SET index symbol `^SET.BK`; 965 observed dates
Chronological split	580 training dates, 194 validation dates, and 194 held-out test dates
Readiness outcome	15 passes, 3 warnings, and 0 failures

The chronological split is shown in Figure 3.1. Training uses 580 dates, validation uses the next 194 dates, and the final 194 dates are held out. The completed performance values reported in this paper come from the validation interval; the held-out interval is not used to support the reported performance comparison. Maintaining temporal order reduces direct future-to-past leakage, although one split cannot remove all backtest-selection risk [2, 1].

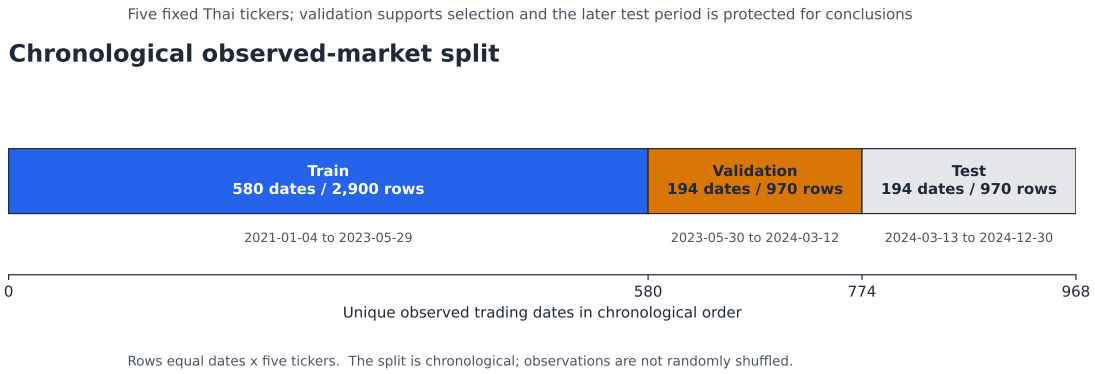


Figure 3.1: Chronological split of the observed five-stock panel. Rows equal dates multiplied by five tickers.

3.3 Features and Leakage Controls

The completed real-OHLCV pilot used a 30-date lookback and 13 configured columns: 1-day, 5-day, and 20-day returns; 20-day volatility; 10-day and 30-day moving-average ratios; 14-day relative strength index; moving-average convergence divergence; Bollinger z-score; 20-day volume ratio; a daily market-return field; a macro-rate-change field; and a sentiment field. The final two fields were constant scaffolds in the frozen table and therefore supplied no historical variation. This paper makes no claim about macroeconomic or sentiment effects.

Features were computed from information available on or before the current date. Duplicate date–ticker keys, missing configured columns, non-finite values, and chronology violations were treated as failed inputs. The historical pilot and the later five-algorithm smoke used related but not identical feature configurations; their results are therefore not combined into one algorithm ranking.

3.4 Portfolio Environment, Algorithms, and Baselines

The environment is a Gymnasium-compatible long-only portfolio allocator. At each date, the state contains the current lookback tensor and portfolio information, while the action is a continuous vector of target portfolio weights. The environment starts with 1,000,000 units of cash, applies a transaction-cost rate of 0.001, and records portfolio value, daily net return, turnover, drawdown, cash weight, and per-ticker allocation. The training reward is

$$r_t = R_t - 0.05T_t - 0.10D_t, \tag{3.1}$$

where R_t is net portfolio return, T_t is turnover, and D_t is the drawdown penalty. The reward guides optimization; portfolio performance is evaluated with separate metrics.

The completed performance pilot trained PPO and A2C and compared them with PTT and ADVANC buy-and-hold, equal weighting, and mean variance. Equal weighting provides a transparent $1/N$ benchmark [10], while mean variance represents classical portfolio optimization [11]. The storage-safe smoke instantiated PPO, A2C, DDPG, TD3, and SAC using Stable-Baselines3 [9]. Each smoke run was deliberately short and was used only to test execution and artifacts.

3.5 Portfolio Metrics

For portfolio value V_t, daily net return r_t, and n evaluated trading dates, the completed pilot metrics are defined in Table 3.3. Annualized volatility uses the population standard deviation and 252 trading periods per year.

Table 3.3: Descriptive portfolio metrics used in the completed pilot.

Metric	Computation	Interpretation
Cumulative return	$C = V_T/V_0 - 1$	Net growth over the evaluated interval
Annualized return	$A = (1 + C)^{252/n} - 1$	Compounded return scaled to 252 periods
Annualized volatility	$\sigma_a = \mathrm{sd}_0(r_t)\sqrt{252}$	Dispersion of daily net returns
Reported return-to-volatility ratio	$S^\star = A/\sigma_a$	Study-defined risk-adjusted ratio; larger is better
Maximum drawdown	$\min_t(V_t/\max_{s\le t} V_s - 1)$	Worst peak-to-trough loss; less negative is better

The quantity $S^\star$ is related to the Sharpe ratio [12], but it is not identical to Sharpe's mean differential-return definition. The distinction is retained throughout the paper. Because the pilot contains one validation split, the analysis is descriptive: exact values are reported without confidence intervals, bootstrap resampling, or multiplicity-adjusted hypothesis tests.

3.6 HPC Calibration and Deterministic Replay

BistKA uses SLURM for shared-cluster scheduling; SLURM's design and resource-management role are described by Yoo, Jette, and Grondona [15]. The completed cal-

ibration evaluated 1, 2, 4, 8, and 16 subprocess vector environments. Each setting executed 4,096 training timesteps on the observed training split with seed 42 and three independent repetitions. The recorded outcomes were samples per second and sampled concurrent process-tree resident memory.

The selected environment count was the setting whose median throughput was at least 95% of the maximum median throughput, with lower memory preferred when performance was comparable. A separate deterministic-replay experiment launched two independent subprocesses with the same seed, runtime versions, resolved configuration, and observed training data. Each subprocess generated a 549-step fixed-random-action environment trajectory. Determinism was accepted only if the trajectory hashes, step counts, provenance hashes, and runtime versions matched. This protocol tests environment replay under one configuration; it does not claim bitwise-deterministic neural-network training.

3.7 Storage-Safe Five-Algorithm Smoke

The completed storage-safe smoke used one vector environment and 2,048 training timesteps for each of PPO, A2C, DDPG, TD3, and SAC. The scheduler requested two CPUs and 8 GiB of memory. Each run was required to produce a resolved configuration, split summary, final model, metrics, equity curve, actions, trial metadata, complete experiment manifest, and checksum evidence.

The retention policy suppressed periodic checkpoint writes while retaining the final model. It also created an atomic retention receipt and SHA-256 sidecar. Acceptance required one complete manifest and one retention receipt per algorithm, matching checksums, final models, and zero paths classified as periodic checkpoints. This artifact design follows the general principle that computational claims should remain linked to exact inputs, configurations, and outputs [16, 17].

Storage was measured from complete returned file inventories. The checkpoint-writing smoke inventory contained 275,209,516 bytes, and the storage-safe inventory contained 137,777,145 bytes. The reduction was calculated as

$$100\left(1-\frac{137{,}777{,}145}{275{,}209{,}516}\right)=49.94\%. \tag{3.2}$$

Chapter 4

DISCUSSION

4.1 Summary of Completed Observations

Table 4.1 separates the financial pilot result from the engineering results. This distinction is essential: the validation pilot compares portfolio behavior, whereas the HPC experiments measure throughput, repeatability, execution, and storage.

Table 4.1: Summary of completed observations.

Experiment	Main observation	Interpretation
Real-OHLCV validation pilot	PTT buy-and-hold ratio 0.405480; A2C -0.434028; PPO -0.439532	No DRL outperformance in this single split
Vector-environment calibration	One environment: 353.0773 samples/s; 16 environments: 82.9607 samples/s	Added subprocesses reduced throughput
Deterministic replay	Identical 549-step hashes in two subprocesses	Environment replay matched under the tested configuration
Five-algorithm smoke	Five complete manifests, models, and receipts; zero checkpoint paths	All five algorithm paths satisfied the artifact contract
Storage comparison	275,209,516 versus 137,777,145 bytes	Storage-safe retention reduced returned output by 49.94%

4.2 Historical Pilot Results

The completed real-OHLCV pilot used one chronological validation split. Table 4.2 reports the preserved values. PTT buy-and-hold had both the highest cumulative return and the highest reported annualized return-to-volatility ratio. PPO and A2C had negative cumulative returns and ratios below the equal-weight portfolio. Mean variance performed worst on the reported ratio.

Table 4.2: Real-OHLCV pilot results on one chronological validation split. Returns are decimals.

Policy	Cumulative return	Reported ratio $S^\star$	Maximum drawdown
PTT buy-and-hold	0.043523	0.405480	-0.103448
ADVANC buy-and-hold	0.009999	0.097452	-0.098919
Equal weight	-0.030573	-0.416554	-0.103240
A2C	-0.038061	-0.434028	-0.120042
PPO	-0.035900	-0.439532	-0.066219
Mean variance	-0.099682	-1.097376	-0.166737

Figure 4.1 emphasizes the signed risk-adjusted comparison. PPO's maximum drawdown (-0.066219) was smaller in magnitude than the equal-weight and A2C drawdowns, but its cumulative return and reported ratio were worse. This is a narrow risk observation, not evidence of superior overall performance.

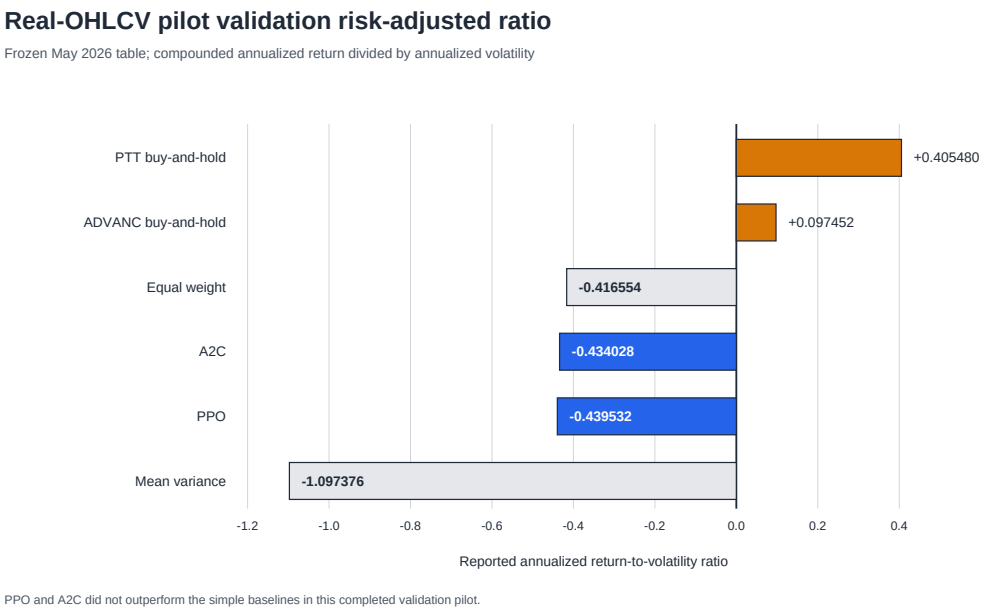


Figure 4.1: Reported annualized return-to-volatility ratios in the completed historical validation pilot. The values come from one preserved validation table.

The simplest interpretation is that model complexity did not create an advantage automatically in this sample. The result does not establish that DRL is ineffective in general, because the cohort is small, the comparison uses one split, and PPO/A2C outcomes depend on their particular training configuration. It does establish that the strongest observed result in this pilot belonged to a simple buy-and-hold baseline.

4.3 One Vector Environment Was Fastest and Most Memory-Efficient

Increasing vector environments reduced throughput monotonically and increased maximum sampled process-tree memory, as shown in Table 4.3. One environment achieved the highest median throughput, 353.0773 samples per second. Sixteen environments achieved 82.9607 samples per second, approximately 76.5% below the one-environment result, while maximum peak resident memory rose from 600.711 to 8,737.996 MiB.

Table 4.3: BistKA vector-environment calibration with three repeats per setting.

Vector environments	Median samples/s	Maximum peak RSS (MiB)
1	353.0773	600.711
2	257.0366	1,641.160
4	212.1034	2,655.523
8	141.2985	4,680.582
16	82.9607	8,737.996

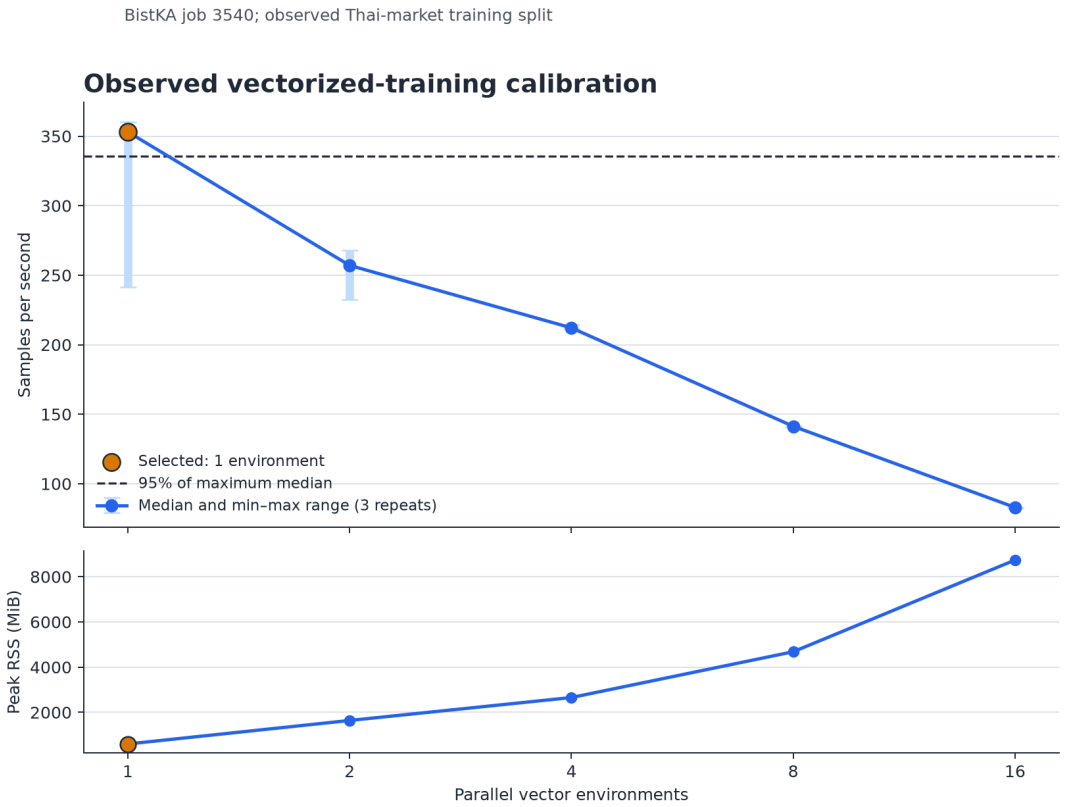


Figure 4.2: Observed throughput and sampled peak resident memory across vector-environment settings. Throughput error ranges show the minimum and maximum of three repeats.

The likely mechanism is overhead: process creation, inter-process communication, and duplicated environment state outweighed useful parallel work for a five-ticker environment. The result demonstrates why a large HPC allocation should be calibrated rather than filled automatically. For this workload, the evidence-supported configuration was `num_envs=1`.

The separate replay experiment strengthens this resource choice at the environment level. Both subprocesses produced 549 steps and the same SHA-256 trajectory

digest. Data, configuration, code, provenance, and runtime records also matched. The observation is deliberately narrow: it supports deterministic environment replay under the tested seed and software stack, not deterministic training across all algorithms or hardware.

4.4 Five Algorithms Satisfied the Storage-Safe Artifact Contract

The completed smoke job executed PPO, A2C, DDPG, TD3, and SAC through the same storage-safe path. Scheduler and artifact outcomes are summarized in Table 4.4. The job completed normally in 2,000 seconds with exit code `0:0`. All five algorithm directories contained complete manifests, final models, and checksum-bound retention receipts, and no periodic checkpoint paths were found.

Table 4.4: Observed scheduler accounting and artifact validation for the five-algorithm smoke.

Field	Observed value
Job name and partition	`drl_safe_smoke`; `compute-devel`
State and exit code	`COMPLETED`; `0:0`
Elapsed time	2,000 seconds = 00:33:20
Allocated resources	2 CPUs; 8 GiB requested memory
Total CPU and maximum batch RSS	32:33.567; 809680K
Algorithms	PPO, A2C, DDPG, TD3, SAC
Accepted artifacts	5 complete manifests; 5 final models; 5 retention receipts
Rejected artifact condition	0 periodic checkpoint paths

This result verifies software-path coverage and artifact completeness. It does not rank algorithms because each smoke run was a single short 2,048-timestep execution with an algorithm-specific sampled configuration. Financial metrics from these runs are therefore excluded from the performance discussion.

4.5 Storage-Safe Retention Halved Returned Output

The checkpoint-writing smoke returned 262.460 MiB, while the storage-safe smoke returned 131.395 MiB. Figure 4.3 displays both complete inventories from a zero baseline.

The 49.94% reduction occurred even though the storage-safe run retained all final models and added manifest, receipt, and checksum evidence.

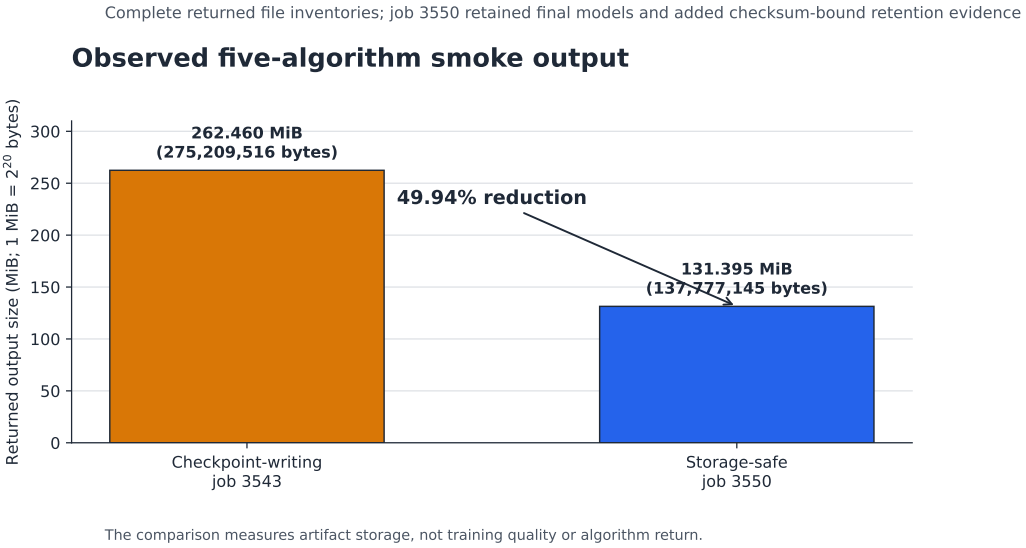


Figure 4.3: Complete returned output inventories before and after storage-safe retention. The comparison measures artifact storage, not investment performance.

Suppressing redundant periodic checkpoints at write time was more effective than retaining them and deleting them afterward. The result is operationally significant because it reduced filesystem pressure without removing the final trained model or the evidence needed to identify the run. The measured percentage applies to these two smoke inventories and should not be interpreted as a universal per-trial reduction.

4.6 Financial and Computational Interpretation

The completed evidence supports two different conclusions. Financially, the historical pilot did not show PPO or A2C outperforming the strongest transparent baseline. Computationally, the environment could be calibrated, replayed reproducibly under fixed conditions, and executed across five algorithm classes with a validated artifact contract. HPC was significant not because more processors improved the result, but because the cluster made repeated measurement, scheduler accounting, and provenance capture practical.

This distinction also clarifies what reproducibility means. Matching environment trajectories, manifests, and checksums reduce the risk of silent code or configuration drift. They do not reduce uncertainty in future market returns, remove sampling bias from a five-stock cohort, or turn a smoke test into a financial comparison. A repro-

ducible negative or inconclusive financial result remains scientifically valuable because it prevents computational complexity from being mistaken for economic evidence.

4.7 Limitations

The interpretation is constrained by the following completed-study limitations:

1. **Fixed cohort.** Five selected equities are not a survivorship-free representation of the whole Thai market.

2. **Single validation split.** The pilot provides descriptive metrics without repeated holdouts, confidence intervals, or multiplicity-adjusted inference.

3. **Configuration difference.** The historical PPO/A2C pilot used an earlier 13-feature configuration, whereas the five-algorithm smoke used a later observed-market configuration. Their financial outputs are not pooled.

4. **Constant context fields.** The macro-rate-change and sentiment columns in the pilot configuration were constant scaffolds and cannot support economic-effect claims.

5. **Preserved table rather than rerun.** The original pilot run directory was not present in the current local bundle; the reported pilot values were taken from the frozen accepted results table and were not regenerated.

6. **Narrow deterministic test.** Matching 549-step random-action trajectories verify environment replay under one runtime, not bitwise-identical model training.

7. **Smoke-test purpose.** Each of the five storage-safe algorithm runs was short and singular. These runs establish executability and artifacts, not relative investment performance.

8. **Storage comparison scope.** The 49.94% reduction is an observation from two complete smoke inventories and may vary with model settings or retention policy.

Chapter 5

CONCLUSION

5.1 Summary of Findings

The completed real-OHLCV pilot did not show PPO or A2C outperforming the strongest simple strategy. PTT buy-and-hold achieved a cumulative return of 0.043523 and a reported annualized return-to-volatility ratio of 0.405480. PPO and A2C produced negative cumulative returns and negative ratios on the same validation interval. PPO exhibited a smaller maximum drawdown than several comparators, but that isolated feature did not offset its weaker return and risk-adjusted result.

The HPC experiments produced clear engineering findings. One vector environment delivered the highest median throughput and the lowest measured process-tree memory among the five settings. Two independent subprocesses generated identical 549-step environment trajectory hashes. A single storage-safe job then completed PPO, A2C, DDPG, TD3, and SAC with five complete manifests, five final models, five checksum-bound retention receipts, and zero periodic checkpoint paths. Compared with the checkpoint-writing smoke, returned output fell by 49.94%.

5.2 Scientific Conclusion

For the completed five-stock pilot, added DRL complexity did not produce superior validation performance relative to the strongest buy-and-hold baseline. The evidence is limited to one chronological validation split and therefore supports a pilot-level negative finding rather than a general statement about Thai equities or DRL.

HPC was nevertheless scientifically significant. Its main contribution was not raw parallelism: excessive vectorization made the tested workload slower and more memory-intensive. Its contribution was controlled measurement and auditability.

BistKA enabled repeatable resource calibration, independent environment replay, scheduler accounting, and artifact-preserving execution across five DRL algorithms. The study therefore demonstrates that careful HPC design can strengthen the reproducibility and storage efficiency of financial machine-learning experiments even when the observed investment result is negative.

REFERENCES

[1] D. H. Bailey, J. M. Borwein, M. López de Prado, and Q. J. Zhu, "The Probability of Backtest Overfitting," *Journal of Computational Finance*, vol. 20, no. 4, pp. 39–69, 2017. doi:10.21314/JCF.2016.322.

[2] V. Cerqueira, L. Torgo, and I. Mozetič, "Evaluating Time Series Forecasting Models: An Empirical Study on Performance Estimation Methods," *Machine Learning*, vol. 109, pp. 1997–2028, 2020. doi:10.1007/s10994-020-05910-7.

[3] X.-Y. Liu, H. Yang, Q. Chen, R. Zhang, L. Yang, B. Xiao, and C. D. Wang, "FinRL: A Deep Reinforcement Learning Library for Automated Stock Trading in Quantitative Finance," *Deep Reinforcement Learning Workshop, NeurIPS 2020*, 2020, arXiv:2011.09607. https://arxiv.org/abs/2011.09607.

[4] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," arXiv:1707.06347, 2017. https://arxiv.org/abs/1707.06347.

[5] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous Methods for Deep Reinforcement Learning," in *Proceedings of the 33rd International Conference on Machine Learning*, PMLR, vol. 48, pp. 1928–1937, 2016. https://proceedings.mlr.press/v48/mniha16.html.

[6] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous Control with Deep Reinforcement Learning," arXiv:1509.02971, 2015. https://arxiv.org/abs/1509.02971.

[7] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing Function Approximation Error in Actor-Critic Methods," in *Proceedings of the 35th International Conference on Machine Learning*, PMLR, vol. 80, pp. 1587–1596, 2018. https://proceedings.mlr.press/v80/fujimoto18a.html.

[8] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor," in *Proceedings of the 35th International Conference on Machine Learning*, PMLR, vol. 80, pp. 1861–1870, 2018. https://proceedings.mlr.press/v80/haarnoja18b.html.

[9] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable Reinforcement Learning Implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021. https://www.jmlr.org/papers/v22/20-1364.html.

[10] V. DeMiguel, L. Garlappi, and R. Uppal, "Optimal Versus Naive Diversification: How Inefficient Is the $1/N$ Portfolio Strategy?" *Review of Financial Studies*, vol. 22, no. 5, pp. 1915–1953, 2009. doi:10.1093/rfs/hhm075.

[11] H. Markowitz, "Portfolio Selection," *The Journal of Finance*, vol. 7, no. 1, pp. 77–91, 1952. doi:10.1111/j.1540-6261.1952.tb01525.x.

[12] W. F. Sharpe, "The Sharpe Ratio," *The Journal of Portfolio Management*, vol. 21, no. 1, pp. 49–58, 1994. doi:10.3905/jpm.1994.409501.

[13] S. Holm, "A Simple Sequentially Rejective Multiple Test Procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979. https://www.jstor.org/stable/4615733.

[14] B. Efron, "Bootstrap Methods: Another Look at the Jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979. doi:10.1214/aos/1176344552.

[15] A. B. Yoo, M. A. Jette, and M. Grondona, "SLURM: Simple Linux Utility for Resource Management," in *Job Scheduling Strategies for Parallel Processing*, Lecture Notes in Computer Science, vol. 2862, pp. 44–60, Springer, 2003. doi:10.1007/10968987_3.

[16] G. K. Sandve, A. Nekrutenko, J. Taylor, and E. Hovig, "Ten Simple Rules for Reproducible Computational Research," *PLOS Computational Biology*, vol. 9, no. 10, e1003285, 2013. doi:10.1371/journal.pcbi.1003285.

[17] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep Reinforcement Learning That Matters," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018. doi:10.1609/aaai.v32i1.11694.